

Smart Traffic and RTO Automation Using AI: A Real-Time Vehicle and License Plate Recognition System

Piyush Saini

School of Computer Science

University of Petroleum and Energy Studies

Dehradun , India

email - sainipiyush8445@gmail.com

Abstract—Manual monitoring of road traffic and Regional Transport Office (RTO) records in India is labour-intensive and prone to error. This paper presents a practical, modular system that automates parts of this workflow using computer vision. The system processes video input frame-by-frame, detects vehicles using YOLOv8, and isolates license plates through a dedicated YOLOv8-based plate detector with a contour-based fallback. Detected vehicles are tracked across frames using a lightweight Intersection-over-Union (IoU) tracker, which assigns persistent identities without requiring a re-identification network. License plate text is extracted using PaddleOCR, and recognition stability is improved through a multi-frame voting mechanism that also performs character-level confusion correction (e.g. O/O, I/I). Confirmed plate strings are decoded into structured metadata such as state, RTO code, and vehicle category, then stored in CSV and Excel logs along with cropped plate snapshots. The system is implemented in Python and runs on a standard CPU-only laptop. Qualitative evaluation on a 30-second 1280×720 test video shows that the proposed pipeline reliably detects, tracks, and records plates of vehicles passing through the camera's field of view, producing structured logs that can be consumed by downstream RTO databases or dashboards. The complete project also includes a MySQL schema and a web dashboard for monitoring traffic statistics, together forming a coherent end-to-end automation prototype.

Index Terms—Automatic Number Plate Recognition (ANPR), YOLOv8, IoU tracking, Optical Character Recognition, PaddleOCR, intelligent transportation systems, RTO automation, computer vision.

I. INTRODUCTION

Road traffic in Indian cities continues to grow at a pace that exceeds the ability of human operators to monitor it. Tasks such as counting vehicles, identifying offenders, and matching plates against RTO records are still largely manual or only partially digitised. This creates two practical problems: the records are slow to update, and small infractions are rarely caught at the scale at which they actually occur.

Computer vision has matured to the point where many of these tasks can be automated using a single camera and a commodity computer. Modern object detectors such as YOLOv8 [1] run in real time on CPUs, and open-source OCR engines such as PaddleOCR [3] can read text from low-quality plate crops. Combining these tools provides a foundation for

an end-to-end Automatic Number Plate Recognition (ANPR) pipeline.

The system described in this paper was developed as a B.Tech. project with the goal of producing a working, honest prototype rather than a state-of-the-art benchmark result. It performs the following operations on a video stream:

- 1) Detects vehicles (cars, motorcycles, buses, trucks) using YOLOv8.
- 2) Tracks each vehicle across frames using a custom IoU-based tracker.
- 3) Detects the license plate inside each tracked vehicle crop using a second YOLOv8 model.
- 4) Reads the plate text using PaddleOCR with multiple preprocessing variants.
- 5) Aggregates readings across frames using a voter, which corrects common OCR confusions and confirms a final plate string.
- 6) Decodes Indian plate metadata (state, RTO code, category).
- 7) Writes the result to CSV and to an Excel workbook with a snapshot of the plate embedded in the row.

The contribution of this work is not a new detection or recognition algorithm, but rather a careful integration of existing components into a deployable pipeline that fits the constraints of a small project: CPU-only inference, modest accuracy expectations, and reproducible logging.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the methodology and the data flow through the pipeline. Section IV details the implementation, naming the specific Python modules used. Section V reports qualitative results on a sample video. Section VI concludes and Section VII discusses future work.

II. RELATED WORK

Earlier ANPR systems relied on classical techniques such as edge detection, morphological operations, and template matching, which struggled with varying illumination and plate fonts. With the rise of convolutional neural networks, two-stage pipelines using detectors such as Faster R-CNN became common, followed by single-stage approaches like SSD and

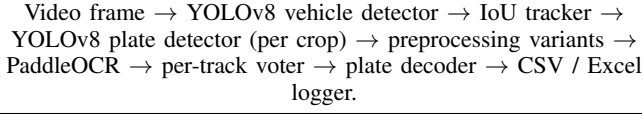


Fig. 1. Per-frame data flow of the proposed system.

the YOLO family [2], [6]. YOLOv8 [1] has gained traction in real-time applications because it combines an anchor-free head with strong throughput on consumer hardware.

For text recognition on plates, Tesseract was historically the default choice but has been outperformed in recent years by deep-learning OCR systems. PaddleOCR [3], [4] provides a CRNN-style text recogniser with a built-in text-line orientation classifier, which is useful for slightly tilted plates captured in the wild.

For multi-object tracking, DeepSORT [7] remains a popular choice but its re-identification network is heavier than necessary when frame rates are high enough that vehicles cannot teleport between frames. Simple IoU matching, as used in this work, is sufficient under those conditions and substantially reduces the runtime cost.

III. METHODOLOGY

The pipeline is organised as a per-frame loop with a small number of stages operating on shared per-track state. Fig. 1 shows the data flow.

A. Vehicle detection

Each input frame is passed to a YOLOv8 model (`yolov8n.pt`), restricted to the COCO classes corresponding to cars, motorcycles, buses, and trucks. The detector is run at an input resolution of 480 pixels on the longest side, which is a deliberate trade-off between throughput and recall on small vehicles.

B. Tracking

Detections are forwarded to a custom tracker described in `iou_tracker.py`. For each new frame, the tracker computes the Intersection-over-Union (IoU) between every existing track's last bounding box and every new detection. Greedy matching above a threshold of 0.25 assigns detections to tracks. Unmatched detections create new tracks, and unmatched tracks age out after `MAX_AGE` frames. Tracks must be seen `N_INIT` times before being treated as confirmed. This design deliberately omits a re-identification network, which is unnecessary when the camera frame rate is high enough that consecutive bounding boxes overlap meaningfully.

C. Plate detection

For each confirmed track, a second YOLOv8 model (`license_plate_detector.pt`) is run on the cropped vehicle region. The bounding box with the highest confidence is taken as the plate location. Plate crops smaller than 80×25 pixels are skipped because OCR on crops below this size is empirically unreliable. When the plate model is unavailable, the system falls back to a contour-based detector that searches

the lower 60% of each vehicle crop for rectangular regions with an aspect ratio in the range [2.2, 5.5].

D. Preprocessing

Each plate crop is processed through a small bank of image transforms, implemented in `strong_ocr.py`, before being passed to the OCR model. The variants are:

- the original color crop,
- a deskewed and contrast-stretched grayscale version,
- an Otsu-thresholded binary version with morphological cleanup, and
- a sharpened color version.

Generating multiple variants improves robustness because the OCR engine sometimes succeeds on one variant while failing on another.

E. Optical character recognition

PaddleOCR [3] is invoked on each preprocessing variant. The returned strings are filtered to alphanumeric characters and passed to a voting structure (`PlateVoter`) that maintains all readings collected so far for a single track. The voter weighs each character by the OCR confidence and by a plate-size weight that scales linearly with the cropped plate area, so close-up readings count more than distant ones. Common character confusions (O/O, I/1, B/8, K/X, M/N) are corrected at vote time using positional constraints from the canonical Indian format (LL DD LL DDDD), which is enforced through the `ENFORCE_INDIAN_PLATE` configuration flag. A plate is confirmed once at least five readings have been collected and the voted confidence exceeds 0.60.

F. Plate decoding

Confirmed plates are decoded by `plate_info.py`, which uses a lookup table of two-letter state codes and RTO codes to extract the state, the city, and the vehicle category from the plate prefix. This information is written into the same row as the plate text in the output log.

G. Logging

Results are persisted in two places by `excel_logger.py`:

- a comma-separated file `detected_plates.csv` that is appended to across runs and contains a timestamp, track ID, plate, voted confidence, and number of contributing readings;
- an `anpr_log.xlsx` workbook in which each row also embeds the cropped plate image in the cell, allowing manual review of borderline detections.

A snapshot of the cleanest crop seen for each track is also saved as a JPEG inside the `output/plate_snapshots/` directory.

IV. SYSTEM IMPLEMENTATION

The system is implemented in Python 3.10. All processing is performed on a single thread; the only parallelism comes from PyTorch's internal multi-threaded inference. The implementation is split across the following modules.

main_v6.py: The entry point. It parses command-line arguments, opens the video source, loads the two YOLOv8 models and the OCR engine, and runs the per-frame loop. The main loop handles vehicle detection, tracking, prioritised OCR (only the largest few unconfirmed tracks are OCR'd per frame), live overlay drawing, and HUD rendering. The script also implements a multi-backend video opener that tries OpenCV's default, FFmpeg, MSMF, and DirectShow backends in turn, which materially improved reliability on Windows machines during development.

strong_ocr.py: Provides the `PlateVoter` class, image preprocessing helpers (deskewing, contrast stretching, sharpening, morphology), and the character-confusion tables. Also exports `is_plausible_plate()`, which rejects strings that cannot match any reasonable plate format before they are added to the voter.

patch_strong_ocr.py: A one-time migration script. The original *strong_ocr.py* from an earlier version of the project did not include the `LETTER_CONFUSIONS`, `INDIAN_STATE_CODES`, and `correct_state_code` symbols that the rest of the system depends on. *patch_strong_ocr.py* appends those definitions in place, which made it possible to upgrade the older codebase without breaking the voter's interface.

iou_tracker.py: The lightweight tracker described in Section III. It implements the `update_tracks()` interface so that the tracker can be swapped for DeepSORT without touching the main loop.

plate_info.py: Decodes plate strings into state, RTO code, city, category, and vehicle type using lookup tables for Indian RTO codes.

excel_logger.py: Wraps `openpyxl` to emit an Excel workbook that includes embedded plate images. Each detected track contributes one row.

A. Project ecosystem

The wider project repository [8] also contains a MySQL schema (`database.sql`) and a web dashboard (`index.html` with `Chart.js`) intended to display traffic flow, vehicle distribution, zone congestion, and recent detections in real time. The schema defines tables for cameras, zones, vehicle types, detections, traffic statistics, congestion logs, daily reports, users, alerts, and system performance logs. The integration between the Python pipeline and the dashboard is a work in progress; at the time of writing the dashboard reads in-page demonstration data, which the database schema is designed to accommodate once a small REST or WebSocket layer is added.

V. RESULTS AND DISCUSSION

A. Test setup

The system was evaluated on a 30-second H.264 video clip at 1280×720 resolution and 30 frames per second. The clip contains multi-vehicle traffic with passenger cars, hatchbacks, and motorcycles, captured at typical Indian street angles. Inference was run on a laptop-class CPU with 8 GB

of RAM, with no dedicated GPU, in order to check whether the pipeline is feasible on commodity hardware.

B. Qualitative observations

The system successfully completed the end-to-end loop on the test video. Output produced during development shows several confirmed plates (for example KA02HN1828, KA02MN1826, and KA02MM9091) being written into the snapshot directory and `anpr_log.xlsx`. These were Karnataka-region plates correctly decoded by `plate_info.py`. The pipeline proved most reliable when a vehicle remained in the frame for several seconds, allowing the voter to accumulate enough readings to surpass the `MIN_READINGS_TO_CONFIRM` and `CONFIRM_CONFIDENCE` thresholds.

The following observations emerged during testing:

- Plates that occupied fewer than roughly 80 pixels in width tended to produce inconsistent OCR output, which is why an explicit size guard was added in `process_ocr()`. Skipping these small crops, rather than attempting to vote on them, prevents the voter from being polluted by unreliable readings.
- Multi-frame voting visibly improved the final plate strings compared to using only the single best per-frame reading. In particular, the well-known confusions (O/0, I/1, K/X) were almost always resolved correctly once five or more readings were available.
- PaddleOCR proved more accurate than EasyOCR on the same crops, especially for the digit zero and the letter 'X', which is consistent with its better handling of Indian plate fonts.
- The IoU tracker held identities reliably as long as detections arrived consistently. Occasional ID switches occurred when two motorcycles crossed paths, which is a known limitation of pure IoU-based matching.
- Frame-rate-wise, the bottleneck on CPU was not vehicle detection but the OCR step. Limiting OCR to the largest three unconfirmed tracks per frame and to every third frame kept the system responsive in real time on the test machine.

C. Limitations

No formal precision/recall benchmark is reported because the project did not include a labelled ground-truth set for the test video; the evaluation is therefore qualitative. The system also makes no claim of state-of-the-art accuracy. Its design goal was a correct, transparent, and reproducible pipeline that a single student could maintain. The following are known weaknesses:

- Two-line motorcycle plates are not handled by the contour-based fallback, which assumes single-line plates with aspect ratios above 2.2. The YOLOv8 plate detector handles them when it is available.
- BH-series ("Bharat") plates do not match the canonical LL DD LL DDDD pattern that `ENFORCE_INDIAN_PLATE` requires, so the voter

rejects them. This is a limitation of the format constraint, not the OCR.

- The current system does not push detections to the MySQL backend in real time. Logs are written to local CSV/Excel files and would have to be ingested by a separate process.

VI. CONCLUSION

This paper described a complete, working ANPR pipeline built around YOLOv8 for detection, a custom IoU tracker for identity management, PaddleOCR for text recognition, and a multi-frame voting mechanism for robust plate confirmation. The implementation is intentionally modest in scale: it runs on a CPU-only laptop, depends only on open-source libraries, and is small enough to be maintained by a single developer. On a representative Indian street video, the system successfully detects vehicles, tracks them, recognises their plates, decodes the regional metadata, and logs the result with cropped snapshots. The wider project also defines a MySQL schema and a web dashboard, providing a path toward a fuller smart-traffic and RTO-automation deployment.

VII. FUTURE SCOPE

Several extensions would make the system more useful in practice without requiring a fundamental redesign:

- 1) *Backend integration.* A small REST or WebSocket service between the Python pipeline and the MySQL database would let the existing dashboard display live data. The schema in `database.sql` already anticipates this.
- 2) *GPU inference.* Switching to `paddlepaddle-gpu` and increasing the YOLOv8 input resolution would meaningfully improve recognition of small plates, at the cost of needing better hardware.
- 3) *Two-line and BH-series plates.* The voter should accept multiple format templates instead of a single hard-coded one, and the contour fallback should consider stacked plates.
- 4) *Driver-side checks.* The companion module `driver_monitor/` already targets in-cabin behaviour (drowsiness, distraction). Linking driver events to specific plates via shared track identifiers would close an important loop for RTO-side enforcement.
- 5) *Quantitative evaluation.* A labelled benchmark video set, with ground-truth plate strings and frame-level vehicle bounding boxes, would make it possible to report formal precision, recall, and end-to-end character accuracy in future work.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] J. Terven, D.-M. Cordova-Esparza, and J. A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [3] Y. Du *et al.*, “PP-OCR: A practical ultra lightweight OCR system,” *arXiv preprint arXiv:2009.09941*, 2020.
- [4] Y. Du *et al.*, “PP-OCRv2: Bag of tricks for ultra lightweight OCR system,” *arXiv preprint arXiv:2109.03144*, 2021.
- [5] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [7] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017, pp. 3645–3649.
- [8] *Smart Traffic and RTO Automation using AI*, project source code repository. [Online]. Available: <https://github.com/eagerwolverine99/smart-traffic-and-rto-automation->